# MapShift: Controlled Post-Intervention Evaluation for Embodied World Models

**Aarav Sinha**[1,2]
[1]Johns Hopkins University, Baltimore, MD, USA
[2]Eon Systems, San Francisco, CA, USA
aarav@eon.systems

## Abstract

Oftentimes, an agent may navigate successfully in a familiar world yet struggle to apply that experience when the world changes. To study this distinction, we introduce MapShift, a benchmark for controlled post-intervention evaluation (CPE). After reward-free exploration, we change the world's metric, topology, dynamics, or semantics, then test the agent in the changed world and compare each world edit to its original. In a 24-motif reference benchmark, MapShift reports performance by intervention family, task, and severity. Unchanged-world evaluation reduces the measured advantage of belief update from $0.304$ to $0.102$ for topology and from $0.724$ to $0.102$ for semantics. Additionally, we find a similar effect in held-out ProcTHOR houses. Across 371 contributing houses, local belief update improves success over stale-map reuse by $+0.532$ after target relocation (95% CI $[0.501, 0.562]$), while the paired gap is essentially zero in unchanged houses. When using an RGB-input recurrent state-space model (RSSM) with a fixed planner, grounded full repair improves success over grounded frozen belief by $+0.218$ ($[0.197, 0.242]$) under CPE. This difference ultimately reverses system preference: the advantage flips from $0.041$ for grounded full repair to $0.224$ for RSSM-only full repair, showing that it is actually superior. These results show that controlled changes can reveal differences in belief updating that familiar-world performance leaves unresolved, with far-reaching consequences for evaluation protocols in robotics and embodied intelligence.

## 1 Introduction

An agent that has explored a house may reach a target by returning to its remembered location. If the target moves before the test, the agent must also use new observations to revise what it remembers. In an unchanged house, however, a system that can revise its map and one that simply reuses it can perform similarly. Most embodied evaluations measure behavior in a sampled environment, without explicitly introducing this change after exploration. As a result, their scores don't separate the ability to store useful experience from the ability to update it and plan from the revised belief, which are distinct problems under partial observability [Kaelbling et al., 1998, Sutton and Barto, 2018].

Here, we introduce controlled post-intervention evaluation (CPE) to make that distinction more measurable. First, an agent explores a base environment $e$ without task reward for $T_{\text{exp}}$ steps. Then, we apply an intervention $I_f(\sigma)$, where $f$ specifies the changed factor and $\sigma$ its severity, and we ask the agent to solve tasks in the resulting environment $e'$. The agent can draw on the state it formed during exploration, along with any permitted local observations. By matching the background world and task conditions and validating the realized edits, we keep track of what each comparison actually changes. Finally, an unchanged-environment control lets us determine which conclusions depend on the intervention (Figure 1).

We begin with a 24-motif reference benchmark in which the edits and their effects can be inspected directly. In this setting, we find that unchanged-world evaluation reduces the measured benefit of

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Evaluations and Datasets Track.

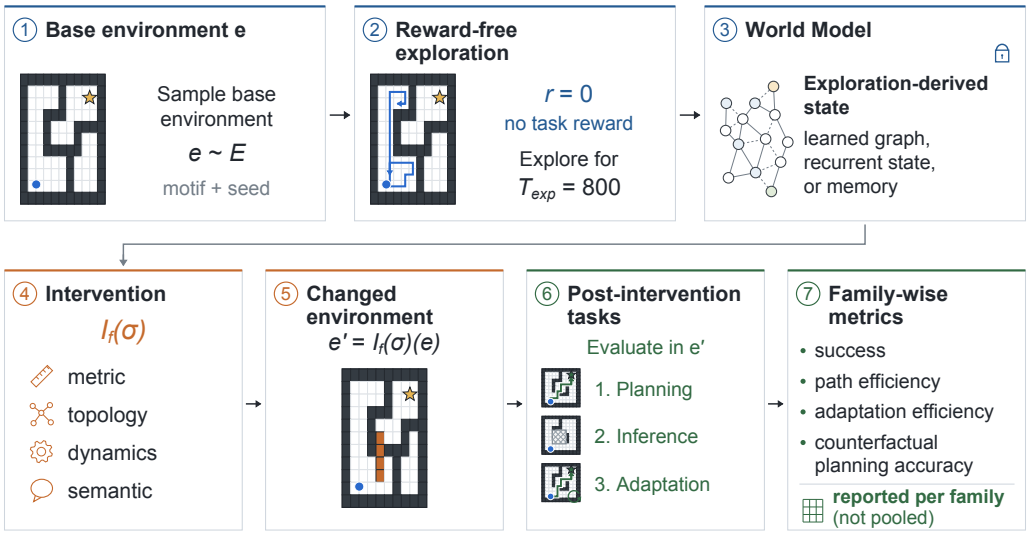



Figure 1: The MapShift protocol, generally summarized. After reward-free exploration in $e$, an intervention from one family at one severity produces a matched environment $e'$, and the agent then performs planning, inference, or adaptation tasks using its stored state and permitted observations. Additionally, validators check the realized edit before results are reported by family, task, and severity.

belief update and changes rankings on planning tasks. Next, we examine semantic and topology edits in held-out ProcTHOR houses, where local belief update gains $0.532$ in success after target relocation, despite having almost no significant advantage in the unchanged control. In addition, controlled variants of an RGB-input RSSM perform differently after intervention, and their relative order reverses. In these comparisons, we evaluate the full process that turns allowed observations into actions or predictions. For an explicit world model, this includes its belief interface and planner, while an end-to-end policy carries its own state throughout the same observation and action interface.

MapShift provides both a protocol for studying post-exploration change and an executable benchmark with the controls and records needed to reproduce it. Reporting results by intervention family, task, and severity, along with validation metrics, lets us trace performance differences directly to specific test conditions. Across the reference benchmark and validated ProcTHOR pairs, our results show that these conditions affect which update mechanisms can be distinguished, and which complete learned system performs better.

## 2 Related work

**Benchmarks for embodied agents.** Benchmarks such as ALE, Gym, DeepMind Control and Lab, Procgen, and MiniGrid provide reproducible settings for studying agents [Bellemare et al., 2013, Brockman et al., 2016, Tassa et al., 2018, Beattie et al., 2016, Cobbe et al., 2020, Chevalier-Boisvert et al., 2023], while Crafter and MineDojo extend this work to open-ended tasks [Hafner, 2022, Fan et al., 2022]. Additionally, embodied environments and benchmarks, including Habitat, AI2-THOR, ProcTHOR, VirtualHome, and ALFRED, support studies of navigation and household activities [Savva et al., 2019, Kolve et al., 2017, Deitke et al., 2022, Puig et al., 2018, Shridhar et al., 2020]. In the context of all these great studies, MapShift specifically asks how experience gained in such a world should be evaluated after a specified change to that same world. For example, in ProcTHOR, we enforce this by verifying actual edits and task solvability, while logging why candidate edits fail.

**World models and reward-free evaluation.** World models often learn predictive structure that can support planning and control [Ha and Schmidhuber, 2018, Hafner et al., 2020, Schrittwieser et al., 2020, Sinha et al., 2026]. Reward-free exploration separates the collection of this information from later reward optimization [Jin et al., 2020, Sekar et al., 2020, Mendonca et al., 2021]. Similarly,

WorldTest (implemented in AutumnBench) tests what an agent learned through planning, inference, and change detection after reward-free interaction [Warrier et al., 2025]. MapShift specifies the family and severity of each intervention and validates the edit, then compares changed and unchanged evaluation on the same generated conditions. Then we can examine how performance depends on the changed factor, such as topology or semantics, alongside the downstream score (See Appendix A for more detail about how this comparison is implemented).

**Interventions, shift, and documentation.** Interventions and counterfactual reasoning also play a role in causal reinforcement learning [Lu et al., 2018, Buesing et al., 2019, Rezende et al., 2020]. Here, the edits are known operations on an environment generator, and are used to study robustness, instead of inferring causal effects from observational trajectories. Additionally, WILDS examines performance under distribution shift [Koh et al., 2021], and studies of benchmark rankings have shown that conclusions can depend on how an evaluation is defined [Maier-Hein et al., 2018, Perlitz et al., 2024]. For this reason, we report results by intervention family, task, and severity. In addition, as per work on dataset and model documentation [Gebru et al., 2021, Mitchell et al., 2019], we record edits, rejected proposals, task solvability, and provenance of each result.

## 3 Controlled post-intervention evaluation

**Protocol.** Let $\mathcal{E}$ denote a distribution of environments, $\mathcal{F}$ the intervention families, and $\Sigma_f$ the severity levels for family $f$. A system $M$ explores $e \sim \mathcal{E}$ through observations and actions without downstream task reward, producing the state

$$z_M = \mathrm{Explore}(M, e, T_{\mathrm{exp}}). \tag{1}$$

Depending on the system, $z_M$ represents its internal state representation, while the no-exploration control uses an empty state. After exploration, an intervention changes one family $f \in \mathcal{F} = \{\mathrm{metric}, \mathrm{topology}, \mathrm{dynamics}, \mathrm{semantic}\}$ at severity $\sigma \in \Sigma_f$:

$$e' = I_f(\sigma)(e). \tag{2}$$

The system carries $z_M$ into $e'$ and receives a task $q \sim Q_c(e, e')$ from a planning, inference, or adaptation class $c \in \mathcal{C}$. Its post-change response is scored as

$$S(M; e, f, \sigma, c, q) = \mathrm{Perf}(M, z_M, e', q)\,. \tag{3}$$

The population score averages over environments, intervention families, severities, and tasks:

$$\mathrm{CPE}(M) = \mathbb{E}_{e\sim\mathcal{E},\, f\sim\mathcal{F},\, \sigma\sim\Sigma_f,\, c\sim\mathcal{C},\, q\sim Q_c(e,e')} \left[S(M; e, f, \sigma, c, q)\right]. \tag{4}$$

Our main analysis separates this expectation by intervention family:

$$\mathrm{CPE}_f(M) = \mathbb{E}_{e,\, \sigma\in\Sigma_f,\, c,\, q} \left[S(M; e, f, \sigma, c, q)\right]. \tag{5}$$

This gives separate measures of performance after changes to geometry, connectivity, dynamics, and semantic binding.

Rather than isolating individual modules, our evaluation encompasses the complete system responsible for constructing and leveraging its internal state, including every component tied to the predictive checkpoint. Within each family, we analyze performance across varying task severities and operational conditions. Although aggregate scores provide a helpful high-level summary, tasks targeting downstream planning alongside inference and adaptation measure distinct capabilities and are therefore reported independently, supported by absolute metrics, paired performance differences, confidence intervals, and filtered edit records.

**Defining an intervention.** To specify what an intervention changes, we can define the environment generator as $G(u, x)$, where $u$ fixes the motif, seed, and layout randomness. The factors than can be editable are $x = (x_{\mathrm{metric}}, x_{\mathrm{topology}}, x_{\mathrm{dynamics}}, x_{\mathrm{semantic}})$, and a family intervention produces

$$e' = G(u, x_{-f}, \phi_f(x_f, \sigma))\,, \tag{6}$$

where $x_{-f}$ denotes the factors outside family $f$ and $\phi_f$ transforms the target factor. In this construction, the pair actually shares the same background world, with validators checking that the intended

edit occurred and the remaining factors stayed fixed. For each episode, we denote the actual change, validation outcomes, and reasons for rejected proposals.

In our finite study, we weight task classes and non-identity severities equally within each family. Additionally, we compare CPE with unchanged-environment, no-exploration, and horizon controls on the same generated grid, and use paired score differences and changes in their method order (see Appendix F for more detail on the estimators and ranking measures).

# 4 Methods

## 4.1 Reference environments and interventions

The reference benchmark is a partially observed navigation world with separate occupancy, transition, and semantic layers. By keeping these layers separate, we can effectively inspect the geometry, connectivity, dynamics, and cue meaning directly, along with each intervention and its checks. The benchmark includes 24 motifs, four intervention families, severities 0–3, three task classes, three samples per class/cell, and an 800 step exploration budget. We split the benchmark by motif, with eight motifs held out for testing (details in Appendix A). We use this world to examine the evaluation procedure, before applying it to held-out 3D scenes in ProcTHOR.

**Environment and observation model.** The reference environment uses $96 \times 96$ occupancy maps at $0.25$m resolution, with $0.25$m forward actions and $15^\circ$ turns. Also, egocentric observations cover a $3.5$m radius and $100^\circ$ field of view, including semantic cues when visible. Finally, each system explores from a navigable start in one episode.

We fix exploration at $T_{\text{exp}} = 800$ steps for all non-reference systems to compare how much useful knowledge they acquire under partial coverage. The health report that our benchmark generates also includes visited-cell and node ratios, path-length distributions, and weak-baseline saturation, therefore sparse coverage in larger motifs remains visible.

**Intervention construction.** Each intervention targets one family while preserving the quantities listed in Table 1. Level 0 leaves the environment unchanged, while levels 1–3 increase the configured magnitude of the edit. Severity levels order the size of changes within each family, but they don't give a common difficulty scale across families. Validators check the realized magnitude and preservation of other factors. Reference solvability and weak-baseline non-saturation provide further checks that the resulting tasks are in actuality truly informative.

Table 1: Reference-benchmark interventions and the quantities they preserve. Severity 0 is the identity condition, while levels 1–3 increase the magnitude of the configured change. Validators check both the target edit, and the factors that should remain fixed.

| Family | Severity parameter | Values | Quantities preserved |
|---|---|---|---|
| Metric | action-gain scale | 1.0, 1.1, 1.25, 1.5 | topology, semantic identities |
| Topology | blocker fraction | 0.0, 0.08, 0.18, 0.32 | semantic identities, action response |
| Dynamics | friction multiplier | 1.0, 0.9, 0.75, 0.6 | geometry, semantic identities |
| Semantic | remap fraction | 0.0, 0.1, 0.25, 0.5 | geometry, action response |

For example, an agent exploring a two-room world may remember a corridor as traversable. A topology intervention then inserts blockers while preserving cue identities and dynamics. The stale planner continues to route through its remembered corridor, whereas a belief updater can revise the map from local observations. Comparing execution in the original world with execution after the corridor changes helps in testing how much this revision contributes to performance.

**Intervention validation.** Validation checks the realized edit and not only the generator's requested operation. For example, metric edits must preserve topology and semantics while topology edits must preserve cue identity and action response as well. Dynamics edits preserve geometry, while semantic edits preserve navigable geometry and reachability. These evaluator-side checks are hidden from the systems, and failed checks are reported *before* invalid pairs are excluded from the analysis.

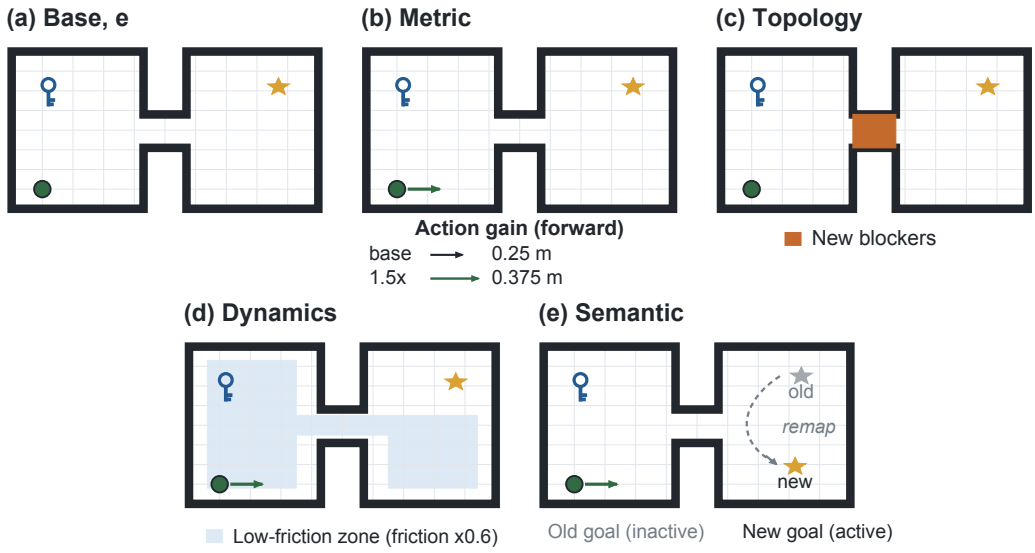



Figure 2: Examples of matched reference worlds. The same two-room world (a) receives a level-3 metric rescaling (b), corridor blockers (c), low-friction dynamics (d), or goal-token remap (e). Motif, seed, key location, and start remain fixed, with validators checking the intended edit and preserved factors. The study also includes severity levels 0–2.

### 4.2 Tasks and baselines

Across comparisons, we use the same policies for the start, goal, visibility, and horizon. Planning covers navigation, rerouting, changed dynamics, and cues, while inference covers topology detection, masked-state prediction, and counterfactual reachability. Finally, adaptation permits a limited amount of post-change interaction before replanning. For reference, planning horizons are 64/96/128 steps, inference horizons are 16/32, and adaptation budgets and horizons are 16/32/64.

The sampler excludes infeasible tasks, queries whose answers do not reflect the intended change, and trivial paths or horizons. We note rejection reasons for each family, severity, and task combination, so that we can distinguish difficult valid conditions from conditions with too few valid tasks.

**Baseline evaluation.** The privileged planner receives the changed world to check solvability and calibrate planning, while the same-environment control executes tasks in the explored world. Our main mechanism comparison pairs a stale-map planner, which keeps its pre-change map fixed, with classical belief update, which revises occupancy and token bindings from local observations. To maintain the model's integrity, the updater receives no intervention label, mismatch location, or global post-change map. Pretrained graph, persistent-memory, relational-graph, and structured-dynamics systems provide learned calibration baselines with fixed hyperparameters. Each record identifies the system, seed, protocol, motif, split, family, severity, task, and bootstrap group (See Appendix B for more details)

In addition, we include a weak heuristic that stores visited cells, visible token bindings, and shallow local traversability. It plans within this remembered structure without global map repair, providing a nontrivial lower baseline that shouldn't saturate the post-change tasks. For more, see Appendix B.

All non-reference methods operate from their exploration-derived state, task specification, and permitted local post-change observations, without access to either the intervention label or the global post-change map.

### 4.3 Scoring

For planning, we record success, executed and reference path lengths, and correctness after the intervention. For record $r$, path efficiency is

$$\eta_r = \begin{cases} \min(\ell_r^*/\ell_r, 1) & \text{if the agent succeeds and } \ell_r > 0, \\ 0 & \text{otherwise,} \end{cases} \tag{7}$$

where $\ell_r^*$ is the shortest feasible path in the evaluation world and $\ell_r$ is the executed path, with a score of zero assigned to failed episodes. We compute post-intervention planning accuracy on tasks whose answers depend on the edit, excluding identity checks, and we keep reference-path measures separate from the mixed family score. This is because the privileged planning reference does not bound performance across all task types.

Inference uses exact-match accuracy for masked-state and post-change reachability queries, and average-rank AUROC with tied ranks for topology detection. Finally, for adaptation, we record recovery over limited post-change interaction and summarize sample efficiency. The study reports 95% intervals from 1,000 bootstrap resamples grouped by environment and model seed, and keeps repeated tasks together within each group. ProcTHOR uses the house-level grouping described in Section 4.5.

### 4.4 Benchmark validity

Before interpreting system performance, we check split separation, coverage by family, severity, and task, rejection reasons, reference solvability, and weak-baseline saturation. The benchmark health report also records paths and horizons, realized intervention magnitudes, preservation of non-target factors, and leakage.

Final-release rejections are counted after internal proposal resampling, so zero rejections means that valid, non-vacuous released tasks cover every condition, rather than that every proposed task passed. Similarly, the severity check verifies that configured change magnitude rises from level 0 to 3; it does not require finite-sample scores to vary monotonically. Weak-baseline saturation or flat success/failure across levels triggers a warning.

Fatal split or task leakage invalidates a result, while diagnostic warnings prompt inspection. Reusable templates are labeled when they reveal no held-out world. In the completed reference run, fatal leakage, warnings, intervention-validator failures, severity-magnitude failures, and final-release task rejections are all zero, with reference solvability of 1.000 (Table A.2).

**Generating the study.** We generate the study from versioned configurations, beginning with a base world, identifier, and manifest for each motif and seed. Systems explore for the configured budget, and learned methods train or load checkpoints identified by baseline, world, seed, and hyperparameters. The pipeline then writes the matched interventions, tasks, validation checks, rejection reasons, and episode records, followed by a benchmark health report that is inspected before interpreting system scores.

**Study design.** The reference study uses the privileged planning diagnostic, weak heuristic, stale-map planner, and classical belief updater to examine the consequences of the evaluation design. We use learned graph, memory, and dynamics baselines as calibration probes to test capacity and representation choices, rather than to build a leaderboard of current agents. The central comparison pairs stale reuse with explicit local update on the same generated grid; the ProcTHOR study then examines corresponding planning schedules and controlled belief interfaces around a trained RGB-input RSSM.

**Seeds and aggregation.** Reference learned baselines use five seeds, and the larger pretrained-graph and persistent-memory rows each cover 2,592 non-identity episodes. Hyperparameters are fixed in versioned configurations before final evaluation, with earlier choices made using validation motifs. We compare CPE, same-environment, and no-exploration conditions on the same grid and report paired update-minus-stale gaps and ranking changes. When resampling, we keep repeated tasks within their environment/model-seed groups.

### 4.5 ProcTHOR study

For the 3D study, we screen 448 houses from the official ProcTHOR test split. Semantic pairs relocate a target within a declared navigable-geometry tolerance, keeping the house, start pose, target identity, and rendering fixed, while topology pairs close an interior door. Each realized edit is checked for geometry, feasibility, and effects on non-target factors. Of 43,633 candidate edits, 5,523 pass across the two families, with rejection reasons retained for the others. The semantic navigation analysis includes 371 contributing houses. Agents receive observations only within 1.5 m, without access to the intervention label, mismatch location, or global post-change map.

The RSSM is trained on 3,260 official training houses and evaluated over three seeds, with no test reachable-lattice hash matching a training hash. Within each seed, variants share the checkpoint, planner, exploration trace, initial recurrent state, tasks, observation interface, and action budget. We pool seeds within house and average the house means. Paired 95% confidence intervals resample houses, retaining all compared systems and seeds from each sampled house together, so that repeated episodes within a scene are not treated as independent observations.

Commands and expected outputs for the smoke test, health audit, result reproduction, and MiniGrid adapter are given in the artifact README and summarized in Appendix G.

## 5 Results

We begin by asking whether a change to the test world reveals how well an agent can update its stored map. The 24-motif reference benchmark provides a setting in which we can inspect each intervention and compare systems that retain or revise their maps under the same conditions. We then examine whether the differences found in this setting also appear for local belief update and learned systems in held-out ProcTHOR houses.

### 5.1 Changes to topology and semantics reveal the benefit of belief update

Before comparing the systems, we confirm that all reference validity checks pass (ie. final task rejections, fatal leakage, leakage warnings, intervention-validator failures, and severity-magnitude failures are all zero). Reference solvability is 1.000, while the weak-heuristic health success proxy is 0.415. Across all 24 motifs, classical belief update scores above stale-map reuse after changes to topology (0.689 versus 0.395) and semantics (0.810 versus 0.178). Stale reuse remains competitive after dynamics changes, however, matching the planning reference at 0.656. In this case, the geometry hasn't changed, so the stored map remains useful.

To determine how much of the update advantage is revealed by the intervention, we compare CPE with unchanged-world evaluation on the eight held-out motifs. For topology, the advantage is roughly *three* times larger under CPE (0.304 versus 0.102); for semantics, it is *seven* times larger (0.724 versus 0.102). Both paired protocol differences are positive (Table 2). Thus, evaluating the agent in a world that stays fixed makes stale-map reuse appear much closer to belief update, even though the two behave differently when the stored information becomes wrong.

Table 2: Belief-update advantages on eight held-out motifs under CPE and unchanged-world evaluation. Each gap is the update score minus the stale-map score over non-identity severities 1–3. Subtracting the unchanged-world gap from the CPE gap gives the protocol delta. Brackets show paired grouped-bootstrap 95% intervals.

| Family | CPE gap | Same-env. gap | Protocol delta |
|---|---|---|---|
| Topology | 0.304 [0.189, 0.424] | 0.102 [0.082, 0.125] | 0.201 [0.095, 0.306] |
| Semantic | 0.724 [0.692, 0.749] | 0.102 [0.080, 0.120] | 0.622 [0.611, 0.631] |

In addition, we find that the effect of the protocol depends on which task is scored. For example, on planning tasks, the unchanged-world and CPE rankings have Kendall $\tau = 0.333$ with two reversals for metric/dynamics tasks, and $\tau = 0.000$ with three reversals for topology tasks at severity 1. Combining tasks into a mixed family score can hide these changes, leaving the overall ranking unchanged despite the differences in planning (Table A.4). For this reason, we report planning, inference, and adaptation separately.

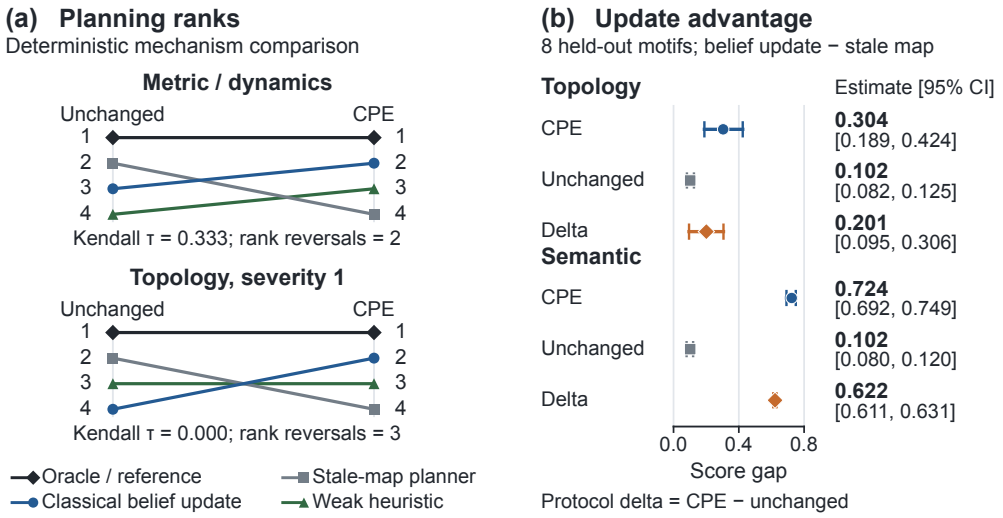



Figure 3: The evaluation protocol changes rankings and measured update advantages in the reference study. (a) Rankings of methods on planning tasks under unchanged-world evaluation and CPE. (b) For held-out motifs, changing the world increases the update-minus-stale gap from 0.102 to 0.304 for topology and from 0.102 to 0.724 for semantics.

Examining the motifs individually, we find that belief update outperforms stale reuse on every held-out topology and semantic motif under CPE. Its advantage increases after intervention on seven of eight topology motifs and all eight semantic motifs (Table A.5). These averages show how consistently the effect occurs across worlds; Table 2 gives the primary paired grouped estimate.

### 5.2 Local belief update improves navigation after target relocation

Having found that changes to the reference world reveal a benefit from updating, we next ask whether the same result holds in 3D houses. We construct ProcTHOR pairs by keeping the house and start fixed while moving a target or closing a door. These pairs allow us to examine local belief update and compare five RSSM variants (Table 3). As shown in Figure 4, target relocation both reveals an update advantage and reverses the preferred belief interface in the learned systems. We use navigation success as the primary metric, together with path efficiency relative to the post-change reference path, assigning zero efficiency to failed episodes.

After target relocation, stale-map reuse succeeds in $0.109$ of episodes, while local belief update succeeds in $0.641$, giving a paired gain of $+0.532$ ($95\%$ CI $[0.501, 0.562]$) across 371 houses. By contrast, the same comparison in unchanged houses yields a paired gap that rounds to $0.000$, with separately rounded means of $0.796$ and $0.797$. Additionally, local update improves path efficiency by $+0.239$ $[0.219, 0.259]$ after relocation, compared with $0.000$ when the world stays fixed.

The updater makes these corrections using local evidence, and in an instrumentation check covering 369 episodes, we find no planning cells outside the observation radius. We then remove exploration memory to test whether the improvement depends on what the agent learned before the change. Success falls from $0.641$ to $0.183$, indicating that both stored experience and local revision contribute to the gain. By comparison, reactive rerouting after door closures reduces the difference in binary success, although updating still improves path efficiency by $+0.056$ $[0.040, 0.072]$.

### 5.3 Post-change evaluation distinguishes repair strategies and reverses system order

We next examine how these findings extend to a learned world model. An RSSM trained from egocentric RGB and pose predicts occupancy and target beliefs for a fixed breadth-first search (BFS) planner, achieving held-out occupancy IoU of $0.56$–$0.58$. Within each seed, we compare five systems using the same checkpoint and evaluation conditions, varying how they construct and update the planning belief without training separate model architectures. In grounded systems, observed cells are passed directly to the planner, whereas the RSSM-only system reconstructs its planning

map from recurrent state. The repair strategies refresh occupancy every eight steps, update the target binding when observations refute it, or combine both operations.

All four grounded variants score $0.859$ to the reported precision when the environment is unchanged, but their scores range from $0.115$ to $0.333$ after target relocation. Compared with frozen belief, full repair improves success by $+0.218$ $[0.197, 0.242]$ and path efficiency by $+0.143$ $[0.128, 0.160]$. Occupancy repair alone reaches $0.116$, while target rebinding reaches $0.225$ and their combination reaches $0.333$. The combined gain of $+0.218$ is larger than the sum of the separate gains (about $+0.11$), indicating that repairing free space and rebinding the target interact. This interaction isn't apparent in the unchanged-world evaluation, where all four variants tie (Table 3).

Table 3: Success on semantic navigation tasks in 371 held-out houses, evaluated over three RSSM seeds. The systems share the checkpoint, planner, episodes, and budget within each seed. Full-repair systems A and B differ in how they construct the planning belief. Brackets show 95% house-bootstrap intervals.

| Evaluated system | Same environment | Semantic CPE |
|---|---|---|
| Grounded, frozen belief | $0.859$ $[0.837, 0.881]$ | $0.115$ $[0.096, 0.135]$ |
| Grounded, occupancy repair | $0.859$ $[0.836, 0.881]$ | $0.116$ $[0.098, 0.135]$ |
| Grounded, target-binding repair | $0.859$ $[0.836, 0.881]$ | $0.225$ $[0.199, 0.251]$ |
| Grounded, full repair (A) | $0.859$ $[0.837, 0.881]$ | $0.333$ $[0.306, 0.360]$ |
| RSSM-only, full repair (B) | $0.818$ $[0.799, 0.838]$ | $0.557$ $[0.527, 0.588]$ |

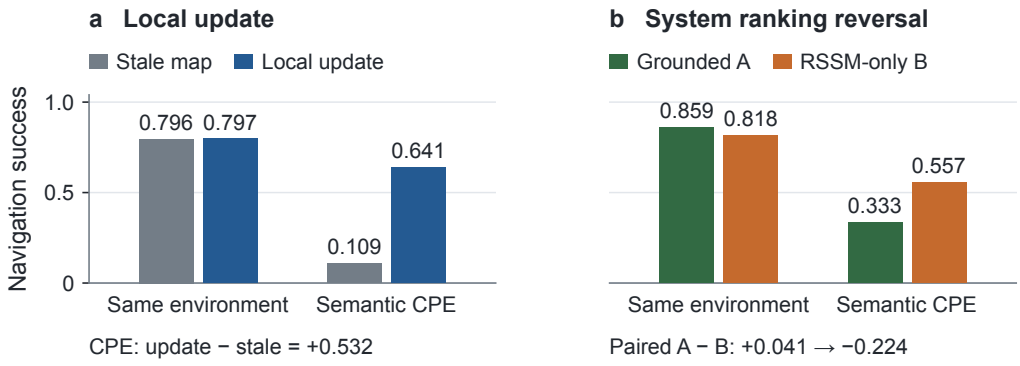



Figure 4: Target relocation reveals differences between systems that appear similar in unchanged ProcTHOR houses. (a) The paired success gap between local update and stale reuse rounds to zero without intervention but reaches $+0.532$ $[0.501, 0.562]$ after relocation. (b) Changing the world reverses the order of the full-repair systems. Unchanged-world evaluation favors grounded belief (A) over RSSM-only belief (B) by $+0.041$ $[0.017, 0.065]$, while CPE favors B by $+0.224$ $[0.192, 0.255]$. Within each seed, A and B use the same checkpoint and planner.

The choice between the full-repair systems also depends on whether the world changes. Direct grounding performs better in unchanged houses, but reconstructing the map from updated recurrent state performs better after target relocation. This reversal occurs with the checkpoint, planner, repair schedule, and information available to the agent paired within each seed.

**Sensitivity to exploration budget and weighting.** Finally, we examine whether the semantic update advantage depends on the amount of exploration or the weighting of test conditions. At 200, 400, 800, and 1600 exploration steps, the advantage remains positive at $+0.267$, $+0.400$, $+0.532$, and $+0.614$, respectively, while unchanged-world gaps stay at most $0.002$. We use 800 steps in the main comparison because unchanged-world success approaches saturation at longer budgets. In addition, the semantic update advantage remains positive across 231 tested severity weightings of validated relocations and 2,000 random reweightings of houses. In both studies, we report family $\times$ task $\times$ severity scores, paired intervals, and rejection counts, using mixed scores only as an additional overview.

## 6 Discussion

An agent that succeeds in a familiar world may have learned a useful map without having learned how to revise it. In both studies that we run, changing the world makes this distinction apparent, as belief update clearly becomes more useful when stored topology or semantics are wrong, while unchanged-world evaluation leaves the systems much closer. The ProcTHOR comparison keeps the house, start, target identity, and rendering fixed as the target moves. Additionally, no-exploration and memory-removal controls establish the role of earlier experience, separating its storage, local revision, and subsequent use in planning.

Validating these comparisons requires confirming that each intervention modified only its intended factor. While reference grid layers can be inspected directly, 3D simulators like ProcTHOR require careful screening, as moving an object or closing a door can introduce unintended side effects on navigability or task feasibility. To account for this, we validate every realized pair against strict geometric tolerances and log all discarded candidate edits. Across the two ProcTHOR families, only 5,523 of 43,633 candidates passed these evaluation checks, which were kept hidden from the agent. Furthermore, because our semantic analysis spans 371 houses, we resample at the house level rather than treating individual episodes as independent samples, preventing statistical overcounting. Consequently, applying this evaluation protocol to new simulators or hardware settings will demand equivalent safeguards for both targeted edits and their side effects.

The learned-model results further show why evaluation must specify how a model's state is used for planning. With the checkpoint and planner fixed within each seed, changing how beliefs are constructed and repaired separates four grounded variants that tie in unchanged houses and reverses the order of the two full-repair variants. In other words, the comparison concerns the model together with the components that turn its state into actions. An end-to-end policy can maintain its own state under the same exploration and intervention procedure, observation limits, and action interface, while paired comparisons and results by family, task, and severity reveal differences hidden by a mixed score for either system type.

As embodied intelligence increasingly relies on learned world models, standard metrics that leave environments unchanged risk masking fundamental flaws in how systems handle change. MapShift highlights that rigorous evaluation requires more than tracking aggregate success, demanding a systematic accounting of how agents perceive, update, and plan when their world changes. **By providing a reproducible protocol for controlled interventions, this work aims to shift how the community measures true agent robustness beyond the confines of familiar testbeds.**

## 7 Limitations and future work

We study a 24-motif reference world, where edits can be inspected directly, and held-out ProcTHOR houses with validated target relocations and door closures in 3D navigation. Future work could include extending these tests to manipulation, language grounding, contact dynamics, and real-life robots, but it's important to keep in mind that such extensions will need checks for coupled changes, rejected proposals, solvability, and observation access in the new simulator or hardware setting. The results also depend on the metric, as path efficiency remains informative when reactive rerouting saturates topology success.

## 8 Conclusion

We introduce MapShift as a benchmark to test whether an agent can use what it learned when the same world undergoes a specified, validated change. In our 24-motif reference study, we find that unchanged-world evaluation reduces belief-update gaps and changes rankings on planning tasks. Target relocation in 371 held-out ProcTHOR houses reveals a $0.532$ local-update advantage that's hidden by the unchanged control, and separates repair choices around one RSSM and reverses the preferred full-repair belief interface. Matching the changed worlds, stating observation limits, and reporting results by intervention family allow readers to inspect how each comparison was made. Our results show that conclusions about a world model can strongly depend on whether it is tested after its world changes.

## Acknowledgments

We thank Prof. William T. Redman for his advice, ideas, and thoughtful discussions.

## References

Charles Beattie, Joel Z. Leibo, Denis Teplyashin, Tom Ward, Marcus Wainwright, Heinrich Küttler, Andrew Lefrancq, Simon Green, Victor Valdés, Amir Sadik, Julian Schrittwieser, Keith Anderson, Sarah York, Max Cant, Adam Cain, Adrian Bolton, Stephen Gaffney, Helen King, Demis Hassabis, Shane Legg, and Stig Petersen. DeepMind Lab. *arXiv preprint arXiv:1612.03801*, 2016.

Marc G. Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47: 253–279, 2013.

Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. OpenAI Gym. *arXiv preprint arXiv:1606.01540*, 2016.

Lars Buesing, Theophane Weber, Yori Zwols, Nicolas Heess, Sébastien Racanière, Arthur Guez, and Jean-Baptiste Lespiau. Woulda, coulda, shoulda: Counterfactually-guided policy search. In *International Conference on Learning Representations*, 2019.

Maxime Chevalier-Boisvert, Bolun Dai, Mark Towers, Rodrigo de Lazcano, Lucas Willems, Salem Lahlou, Suman Pal, Pablo Samuel Castro, and Jordan Terry. Minigrid & miniworld: Modular & customizable reinforcement learning environments for goal-oriented tasks. *arXiv preprint arXiv:2306.13831*, 2023.

Karl Cobbe, Christopher Hesse, Jacob Hilton, and John Schulman. Leveraging procedural generation to benchmark reinforcement learning. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 2048–2056, 2020.

Matt Deitke, Eli VanderBilt, Alvaro Herrasti, Luca Weihs, Kiana Ehsani, Jordi Salvador, Winson Han, Eric Kolve, Aniruddha Kembhavi, and Roozbeh Mottaghi. ProcTHOR: Large-scale embodied AI using procedural generation. In *Advances in Neural Information Processing Systems*, 2022.

Linxi Fan, Guanzhi Wang, Yunfan Jiang, Ajay Mandlekar, Yuncong Yang, Haoyi Zhu, Andrew Tang, De-An Huang, Yuke Zhu, and Anima Anandkumar. MineDojo: Building open-ended embodied agents with internet-scale knowledge. In *Advances in Neural Information Processing Systems*, 2022.

Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64 (12):86–92, 2021.

David Ha and Jürgen Schmidhuber. Recurrent world models facilitate policy evolution. In *Advances in Neural Information Processing Systems*, volume 31, pages 2451–2463, 2018.

Danijar Hafner. Benchmarking the spectrum of agent capabilities. In *International Conference on Learning Representations*, 2022.

Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations*, 2020.

Chi Jin, Akshay Krishnamurthy, Max Simchowitz, and Tiancheng Yu. Reward-free exploration for reinforcement learning. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 4870–4879, 2020.

Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998.

Pang Wei Koh, Shiori Sagawa, Henrik Marklund, Sang Michael Xie, Marvin Zhang, Akshay Balsubramani, Weihua Hu, Michihiro Yasunaga, Richard Lanas Phillips, Irena Gao, Tony Lee, Etienne David, Ian Stavness, Wei Guo, Berton Earnshaw, Imran Haque, Sara M. Beery, Jure Leskovec, Anshul Kundaje, Emma Pierson, Sergey Levine, Chelsea Finn, and Percy Liang. WILDS: A benchmark of in-the-wild distribution shifts. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 5637–5664, 2021.

Eric Kolve, Roozbeh Mottaghi, Winson Han, Eli VanderBilt, Luca Weihs, Alvaro Herrasti, Matt Deitke, Kiana Ehsani, Daniel Gordon, Yuke Zhu, Aniruddha Kembhavi, Abhinav Gupta, and Ali Farhadi. AI2-THOR: An interactive 3d environment for visual AI. *arXiv preprint arXiv:1712.05474*, 2017.

Chaochao Lu, Bernhard Schölkopf, and José Miguel Hernández-Lobato. Deconfounding reinforcement learning in observational settings. *arXiv preprint arXiv:1812.10576*, 2018.

Lena Maier-Hein, Matthias Eisenmann, Annika Reinke, Sinan Onogur, Marko Stankovic, Patrick Scholz, Tal Arbel, Hrvoje Bogunović, Andrew P. Bradley, Aaron Carass, et al. Why rankings of biomedical image analysis competitions should be interpreted with care. *Nature Communications*, 9(1):5217, 2018.

Russell Mendonca, Oleh Rybkin, Kostas Daniilidis, Danijar Hafner, and Deepak Pathak. Discovering and achieving goals via world models. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, pages 220–229, 2019.

Yotam Perlitz, Ariel Gera, Ofir Arviv, Asaf Yehudai, Elron Bandel, Eyal Shnarch, Michal Shmueli-Scheuer, and Leshem Choshen. Do these LLM benchmarks agree? fixing benchmark evaluation with BenchBench. *arXiv preprint arXiv:2407.13696*, 2024.

Xavier Puig, Kevin Ra, Marko Boben, Jiaman Li, Tingwu Wang, Sanja Fidler, and Antonio Torralba. VirtualHome: Simulating household activities via programs. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 8494–8502, 2018.

Danilo Jimenez Rezende, Ivo Danihelka, George Papamakarios, Nan Rosemary Ke, Ray Jiang, Theophane Weber, Karol Gregor, Hamza Merzic, Fabio Viola, Jane Wang, Jovana Mitrovic, Frederic Besse, Ioannis Antonoglou, and Lars Buesing. Causally correct partial models for reinforcement learning. *arXiv preprint arXiv:2002.02836*, 2020.

Manolis Savva, Abhishek Kadian, Oleksandr Maksymets, Yili Zhao, Erik Wijmans, Bhavana Jain, Julian Straub, Jia Liu, Vladlen Koltun, Jitendra Malik, Devi Parikh, and Dhruv Batra. Habitat: A platform for embodied AI research. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 9339–9347, 2019.

Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588: 604–609, 2020.

Ramanan Sekar, Oleh Rybkin, Kostas Daniilidis, Pieter Abbeel, Danijar Hafner, and Deepak Pathak. Planning to explore via self-supervised world models. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 8583–8592, 2020.

Mohit Shridhar, Jesse Thomason, Daniel Gordon, Yonatan Bisk, Winson Han, Roozbeh Mottaghi, Luke Zettlemoyer, and Dieter Fox. ALFRED: A benchmark for interpreting grounded instructions for everyday tasks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10740–10749, 2020.

Aarav Sinha, Oluwajomiloju C. Fasehun, and William T. Redman. Predictive grid cell coding does not require explicit planning. *bioRxiv*, 2026. doi: 10.64898/2026.10.09.757963. URL `https://doi.org/10.64898/2026.10.09.757963`.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.

Yuval Tassa, Yotam Doron, Alistair Muldal, Tom Erez, Yazhe Li, Diego de Las Casas, David Budden, Abbas Abdolmaleki, Josh Merel, Andrew Lefrancq, Timothy Lillicrap, and Martin Riedmiller. Deepmind control suite. *arXiv preprint arXiv:1801.00690*, 2018.

Archana Warrier, Thanh Dat Nguyen, Michelangelo Naim, Moksh Jain, Yichao Liang, Karen Schroeder, Cambridge Yang, Joshua B. Tenenbaum, Sebastian Josef Vollmer, Kevin Ellis, and Zenna Tavares. Benchmarking world-model learning. *arXiv preprint arXiv:2510.19788*, 2025.

# Appendix

## A Additional result tables

Table A.1: Evaluation after reward-free interaction. WorldTest/AutumnBench assesses planning, inference, and change detection, while the MapShift column describes the matched interventions, controls, and reporting choices used here.

| Axis | Derived-test protocol | MapShift CPE protocol |
|---|---|---|
| Changed factor | Tests knowledge through derived tasks, including change detection and changes to dynamics. | Specifies the changed factor (metric, topology, dynamics, or semantics) and its severity. |
| Matched pair | Constructs tests from the environment encountered during reward-free interaction. | Holds motif, seed, start state, and non-target factors fixed across $e$ and $e'$. |
| Failure attribution | Reports performance on planning, inference, and change-detection tasks. | Scores each intervention family separately to show which changes lead to failure. |
| Protocol ablations | Comparisons follow the benchmark's task and evaluation design. | Runs same-environment, no-exploration, and horizon controls on the same grid. |
| Validity checks | Uses benchmark-specific criteria for valid tests. | Reports intervention validation, leakage checks, rejection counts, solvability, and saturation checks. |

Table A.2: Checks of the expanded benchmark. We run these before interpreting model results and include them in the release.

| Check | Value |
|---|---|
| Environments and splits | 24 motifs; 10 train, 6 validation, 8 test |
| Task coverage | 3456 tasks; 0 task rejections |
| Reference solvability | 1.000 |
| Weak-baseline health estimate | 0.415 success proxy; not primary CPE score |
| Fatal leakage / benign warnings | 0 / 0 |
| Intervention validator failures | 0 |
| Severity-magnitude failures | 0 |

Table A.3: MapShift settings used for the main study and its released implementation.

| Component | Value | Component | Value |
|---|---|---|---|
| Map size | $96 \times 96$ | Exploration budget | $T_{\text{exp}} = 800$ |
| Motifs | 24 | Splits | 10 train, 6 val, 8 test |
| Families | metric/topology/dynamics/semantic | Severities | 0, 1, 2, 3 |
| Task classes | planning/inference/adaptation | Task samples | 3 per class/cell |
| Trace-trained seeds | 5 per learned baseline | Bootstrap | 1000 grouped resamples |

Table A.4: Protocol sensitivity within each family. Scores average over task classes, and each comparison changes one evaluation condition while keeping the same generated 24-motif study grid.

| Comparison | Kendall $\tau$ | Rank reversals | Family changes |
|---|---|---|---|
| No exploration vs. reward-free exploration | 1.000 | 0 | 1 |
| Same-environment vs. CPE | 1.000 | 0 | 2 |

These scores average over planning, inference, and adaptation within each family. For planning results alone, see Section 5.1 and Table A.12.

Table A.5: Consistency of the update advantage across motifs. We subtract stale-map scores from belief-update scores within each motif, then average equally over eight test motifs. Table 2 reports the primary paired estimate and grouped interval, while the final column here counts CPE wins and motifs where CPE increases the update advantage.

| Family | Motifs | Mean CPE gap | Mean same-env. gap | Mean protocol delta | BU wins / reduction |
|---|---|---|---|---|---|
| Topology | 8 | 0.336 | 0.102 | 0.234 | 8/8; 7/8 |
| Semantic | 8 | 0.724 | 0.102 | 0.622 | 8/8; 8/8 |

Table A.6: Scores across severities 0–3 in the full run. We show the primary score for each family and two calibration baselines, with severity data and curves for all methods and families included in the release. Validation requires intervention magnitude to increase with severity; the measured scores don't have to change monotonically.

| Baseline | Family | S0 | S1 | S2 | S3 |
|---|---|---|---|---|---|
| Privileged planning ref. | Metric | 0.521 | 0.486 | 0.471 | 0.443 |
| Privileged planning ref. | Topology | 0.521 | 0.552 | 0.750 | 0.740 |
| Privileged planning ref. | Dynamics | 0.521 | 0.654 | 0.656 | 0.657 |
| Privileged planning ref. | Semantic | 0.521 | 0.802 | 0.833 | 0.823 |
| Weak heuristic | Metric | 0.512 | 0.440 | 0.409 | 0.407 |
| Weak heuristic | Topology | 0.486 | 0.566 | 0.444 | 0.516 |
| Weak heuristic | Dynamics | 0.512 | 0.677 | 0.697 | 0.693 |
| Weak heuristic | Semantic | 0.486 | 0.122 | 0.122 | 0.141 |

Table A.7: Mechanism comparison by family under CPE for non-identity severities 1–3. We compare reuse of an unchanged map, local heuristic priors, and explicit belief update, giving equal weight to each severity and task class. The release separately reports reference path lengths and gaps from the reference for planning tasks.

| Method | Metric | Topology | Dynamics | Semantic |
|---|---|---|---|---|
| Privileged planning ref. | 0.467 | 0.681 | 0.656 | 0.819 |
| Stale-map planner | 0.424 | 0.395 | 0.656 | 0.178 |
| Weak heuristic | 0.419 | 0.509 | 0.689 | 0.128 |
| Classical belief update | 0.485 | 0.689 | 0.648 | 0.810 |

## B Baseline details

Table A.8: Deterministic reference baselines. These methods have no trainable parameters and let us check solvability, measure protocol sensitivity, and characterize weak-baseline performance.

| Baseline | Access and stored state | Evaluation behavior | Training |
|---|---|---|---|
| Privileged post-intervention planning reference | Uses privileged access to the evaluation environment to check solvability and provide a planning reference. | Computes shortest paths, exact inference answers, and a reference adaptation curve when the task is solvable. | 0 params; none. |
| Same-environment reference condition | Runs the system in the explored environment without applying the intervention. | Uses the same deterministic planner as the corresponding evaluation condition, providing a control for the protocol. | 0 params; none. |
| Weak heuristic | Remembers visited cells and node ids, visible goal-token bindings, and limited local paths, but does not revise the global map. | Plans orthogonal visible paths through remembered structure and predicts no topology change. Semantic reachability uses stale token bindings, while a task-type rule predicts the masked family. Adaptation follows a capped heuristic curve. | 0 params; none. |
| Stale-map planner | Stores the serialized environment from reward-free exploration and leaves it unchanged after intervention. | Uses the pre-intervention map to plan shortest paths and answer semantic queries, so its score measures the effect of relying on stale exploration memory in the evaluation environment. | 0 params; none. |
| Classical belief update | Remembers an occupancy map, edges, a dynamics signature, and semantic-token bindings from exploration. | Uses local post-change mismatches to update edges, costs, dynamics, and token bindings, then replans within the task/adaptation budget, without receiving the intervention family label or the global post-intervention map. | 0 params; none. |

**Training source.** We train the persistent-memory, relational-graph, and structured-dynamics baselines separately for each base environment and model seed, using data from that instance of exploration. To examine greater model capacity, we pretrain the larger graph world model on generated graphs before CPE evaluation. All evaluation tensors come from the graph obtained during exploration. Node features contain normalized position, local statistics of free, blocked, and semantic observations, role indicators, and global dynamics features. Pairs of nodes are labeled for edge existence, normalized geometric path cost, and normalized traversal cost, while individual nodes are labeled for semantic-token location.

**Optimization and hyperparameters.** We optimize the relational and structured-dynamics baselines with Adam, using a learning rate of 0.01, validation fraction of 0.2, and early-stopping patience of 3. The large persistent-memory run uses 2048 memory slots and a readout width of 672, with Adam learning rate 0.0005, a budget of 120 epochs, and patience 20. Similarly, the pretrained graph world model uses Adam with learning rate 0.0005, a budget of 120 epochs, and patience 20. We fix the hyperparameters in configuration files before final evaluation, with configuration choices intended to use validation motifs only. Test motifs are not used for hyperparameter selection. Each checkpoint is identified by baseline name, environment id, model seed, and hyperparameter hash, and is reused across protocol variants.

**Compute reporting.** The smoke build takes approximately 2–5 minutes on CPU, while the one-command audit takes longer because it also runs the test suite. The full 24-motif study is designed for a single CUDA GPU. On an NVIDIA L4 GPU with 23GB memory, we estimate roughly 30–36 wall-clock hours, with faster runs expected on L40S/H100-class GPUs. These estimates exclude

preliminary and debug runs. Training jobs are small, while most runtime goes to task generation, planning and evaluation loops, bootstrap aggregation, and rendering outputs.

Table A.9: Learned baseline details. The listed trainable parameter counts represent the release configuration, while each run manifest records its exact count, device information, checkpoint, and training summary.

| Baseline | Stored representation and architecture | Supervision and optimization | Size |
|---|---|---|---|
| Pretrained graph world model | Graph model with hidden size 256, 6 message-passing steps, pair width 256, and dynamics width 128. | Pretrained on 4000 generated graphs with 800 validation graphs, then evaluated on the same CPE task grid. Uses a budget of 120 epochs, Adam lr 0.0005, patience 20, and 2592 non-identity evaluation episodes. | $\sim$1.14M params. |
| Persistent-memory world model | Builds the same graph data, then reads each node through learned memory slots that retain exploration state through slot attention. | Uses 2048 memory slots, readout width 672, and the same self-supervised heads, with a training budget of 120 epochs, Adam lr 0.0005, validation fraction 0.2, and patience 20. Evaluation covers 2592 non-identity episodes. | $\sim$1.4M params. |
| Relational graph world model | Encodes node features, then applies 2 message-passing steps over the normalized adjacency matrix obtained during exploration. | Uses the same self-supervised targets, 10 epochs, Adam lr 0.01, validation fraction 0.2, patience 3, and 5 seeds. | 396 params. |
| Structured-dynamics world model | Encodes geometry and dynamics separately. Edge and geometry heads use geometry factors, while the traversal-cost head also receives global dynamics factors. | Uses the same self-supervised targets, 10 epochs, Adam lr 0.01, validation fraction 0.2, patience 3, and 5 seeds, with each prediction head restricted to its specified latent factors. | 504 params. |

# C MiniGrid adapter

We include a MiniGrid-compatible adapter to examine how the protocol transfers to another environment. It replaces the map generator with a MiniGrid-style wrapper while keeping the CPE interface. The adapter samples base worlds deterministically, applies the declared interventions, and validates each resulting pair. It also provides a smoke command, with the option to instantiate an actual `minigrid` environment. Outputs follow the MapShift schema and record the split id, base environment id, intervention family, severity, any rejection reason, reference-solvability status, and validator results. This lets us apply the same evaluation requirements in a different environment.

Table A.10: Design of the MiniGrid adapter, which uses symbolic grids. Visual or language-conditioned wrappers would need the additional checks listed in the right column.

| Component | MiniGrid instantiation | Validators that transfer | Validators needing redesign |
|---|---|---|---|
| Base generator | Uses a seed to sample rooms, doors, keys, walls, lava, and goals, with a motif id in place of the map motif. | Split leakage, motif counts, reachable-state coverage, reference solvability. | For natural-language missions, check the mission templates for leakage. |
| Metric shift | Vary action cost, step budget, turn/forward cost, or grid-edge weights, while keeping walls and object identities fixed. | Topology preservation, object identity preservation, path-cost monotonicity. | Define costs explicitly, since MiniGrid uses discrete actions rather than metric distance. |
| Topology shift | Add or remove doors, walls, blockers, or one-way passages, while preserving object identities and the start/goal distribution. | Connectivity change, unchanged non-target semantics, reference solvability, and task rejection counts. | Check reachability using both the geometry and the keys in the agent's inventory. |
| Dynamics shift | Add slip or stochastic action failure, or change locked-door transitions or lava penalties. | Geometry and semantic preservation, severity-response checks, weak-baseline non-saturation. | When transitions are stochastic, score the reference using repeated rollouts or analytic transition matrices. |
| Semantic shift | Remap goal color/object type, key-door binding, or mission target, while leaving geometry unchanged. | Geometry/topology preservation, target-binding change, non-vacuous task checks. | Check that the wording of language templates doesn't reveal the answer. |
| Tasks | Test navigation after a change, inference about changed doors/keys, masked-cell prediction, and replanning after limited interaction. | Family/task coverage, horizon distributions, final-release rejection accounting. | State whether the agent receives symbolic grids, pixels, missions, or a combination. |

Table A.11: Sanity study for the MiniGrid-compatible adapter. The study uses a symbolic grid wrapper, with optional `minigrid` instantiation; it does not use pixel inputs or vision-language-action models. We repeat the deterministic mechanism comparison from Section 5.1, testing the privileged planning reference, stale-map planner, weak local planner, and classical belief update under CPE, same-environment, and no-exploration controls.

| Quantity | Result |
|---|---|
| Grid and splits | 6 MiniGrid-style motifs, 24 environments; train/val/test = 12/4/8. |
| Matched intervention pairs | 384 pairs covering metric, topology, dynamics, and semantic changes. |
| Health checks | 0 task rejections, 0 validation failures, reference solvability 1.000. |
| Weak baseline | Weak local planner score 0.496; non-saturating. |
| Protocol-sensitive gaps | Belief update exceeds stale-map reuse by 0.313 for topology and 0.750 for semantic shifts under CPE, compared with gaps of 0.000 in both same-environment and no-exploration controls. |
| Ranking effect | Semantic rankings differ between same-environment evaluation and CPE: Kendall $\tau = 0.667$, with one rank reversal. |

Before interpreting scores, we apply the same validity requirements: no leakage or failed validation, rejection reasons reported by cell, solvable reference tasks, and a weak baseline that doesn't saturate. In ProcTHOR, we replace these grid-graph checks with checks of the realized object and door changes, navigable geometry, visibility, and feasibility. Other simulators and hardware need

equivalent checks suited to their environments. The MiniGrid adapter provides a compact way to test the common interface and its validators.

# D Findings across studies

Table A.12 summarizes the reference and ProcTHOR findings, while Table A.11 reports the separate symbolic MiniGrid results. The held-out 3D study extends the comparison to richer observations and learned systems. Together, these findings show how the protocol affects each matched comparison, without implying that every aggregate ranking changes.

Table A.12: Results from the reference and held-out 3D studies, with the evidence supporting each conclusion that depends on the evaluation protocol.

| Standard conclusion | Source | MapShift diagnosis | Why it matters |
|---|---|---|---|
| Same-env task ranking is enough. | 24-motif mechanism diagnostic | Planning rankings change with the protocol: metric/dynamics have $\tau = 0.333$ with two reversals; topology severity-1 planning has $\tau = 0.000$ with three. | The protocol can change method order. |
| Stale maps are competitive. | Mechanism diagnostic | Under CPE, belief update outperforms stale-map reuse on all 8 held-out motifs for topology and semantics. | These changes test the ability to update stored knowledge. |
| Memory capacity is enough. | Capacity runs | Large persistent memory reaches 0.313–0.354, while update-based references score higher in most families. | Greater capacity alone does not replace explicit update. |
| Same-env gaps estimate the intervention effect. | Paired deltas | The belief-update advantage over stale-map reuse is larger under CPE by 0.201 for topology and 0.622 for semantic shifts. | The evaluation condition changes the measured advantage. |
| A high unchanged-house score establishes post-change repair. | 371 ProcTHOR houses | Local update gains $+0.532$ success after semantic relocation, although the gap rounds to zero in the unchanged control. | Post-change tests reveal the value of local update. |
| The unchanged-house choice of belief interface transfers. | Paired RSSM systems | Grounded full repair leads by $0.041$ without a change, whereas RSSM-only full repair leads by $0.224$ after relocation. | The protocol can reverse the preferred system. |

## E ProcTHOR effect sizes

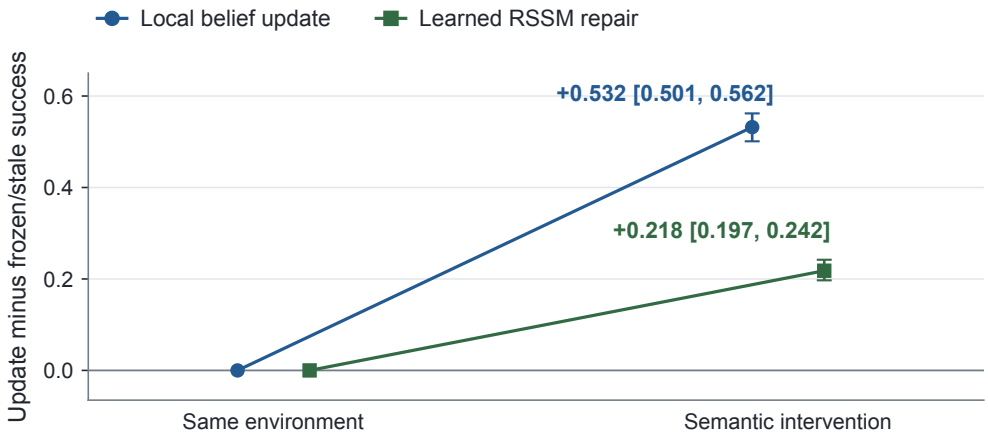



Figure A.1: Paired differences in semantic-navigation success across 371 contributing ProcTHOR houses. In unchanged houses, the gaps round to zero, while CPE separates local update from stale reuse and grounded full-repair RSSM beliefs from grounded frozen beliefs. Brackets give 95% house-bootstrap intervals.

## F Score estimation and protocol comparison

**Estimating the score.** Our study uses finite sets of motifs, environment seeds, severities, tasks, and model seeds. Let $\mathcal{D}_f$ denote the valid episode records for $M$ in family $f$ after task rejection. We estimate the family score as

$$\widehat{\mathrm{CPE}}_f(M) = \frac{1}{|\mathcal{D}_f|} \sum_{r \in \mathcal{D}_f} s_r, \tag{A.1}$$

where $s_r$ is the task-normalized score for record $r$.

**Weighting tasks and severities.** In the primary family-score tables, we use non-identity severities $\Sigma_f^\star = \{1, 2, 3\}$, weighting severity and task class equally:

$$\widehat{\mathrm{CPE}}_f^\star(M) = \frac{1}{3} \sum_{\sigma \in \Sigma_f^\star} \frac{1}{3} \sum_{c \in \{\mathrm{plan,infer,adapt}\}} \frac{1}{|\mathcal{D}_{f\sigma c}|} \sum_{r \in \mathcal{D}_{f\sigma c}} s_r. \tag{A.2}$$

Within each family, severity, and task combination, all records receive equal weight. Severity 0 checks behavior when nothing changes, so we exclude it from non-identity results such as Table 2. In that table, the protocol difference $\Delta_{\mathrm{protocol}}(f) = (\mathrm{BU} - \mathrm{stale})_{CPE,f} - (\mathrm{BU} - \mathrm{stale})_{\mathrm{same},f}$ measures how much more strongly CPE separates belief update (BU) from stale reuse. The privileged reference bounds planning feasibility, but doesn't bound the mixed score across task classes.

**Protocol comparison.** To compare protocols, we change one evaluation condition while keeping the generated grid the same. The same-environment control runs tasks in $e$ instead of $e'$, while the no-exploration control empties $z_M$ and horizon controls vary the task budget. Under protocol $p$, the methods are ordered as

$$\pi_p(f) = \mathrm{rank}_M\left(\widehat{\mathrm{CPE}}_{f,p}(M)\right), \tag{A.3}$$

We compare these orders using Kendall $\tau$, pairwise rank reversals, and changes in the top method or the ordering within each family.

### F.1 Reporting a new implementation

For a new MapShift implementation, researchers must record identifiers for the original and changed worlds, the declared edit and its tolerance, validation results, and rejected proposals before presenting model scores. In addition, they should state which observations and intervention metadata reach the agent, check reference solvability, and report coverage and weak-baseline saturation for each condition. Where possible, starts, tasks, and seeds should be paired across controls, with confidence intervals obtained by resampling independent worlds. Reporting absolute scores and paired differences by family $\times$ task $\times$ severity, together with a mixed summary, lets readers examine whether an effect depends on the protocol, missing tasks, unintended access to the edit, or the aggregation rule.

## G Artifact contents

We release the executable benchmark and generator with code, versioned configurations, schemas, and commands for regenerating split manifests, interventions, task records, health reports, results by family and severity, protocol comparisons, and provenance. The README provides paths for the reproduction routes listed below. Additionally, the ProcTHOR pair manifests, rejection records, and records grouped by house support offline recomputation. Any generated dataset hosted separately should include Croissant core and Responsible-AI metadata.

Table A.13: Commands for reproducing the reference study. Full-study runtimes assume a new run, while tables and figures can be regenerated from existing records in minutes.

| Route | Expected output |
|---|---|
| CPU smoke (2–5 min) | Small records, configs, and sanity tables |
| Health audit | Leakage, rejection, validator, solvability, and coverage report |
| Result reproduction | Table 2, Figure 3, intervals, and assets. A full 24-motif rerun takes roughly 30–36 h on an L4 GPU |
| MiniGrid smoke | 24 environments, 384 pairs, health checks, and solvability 1.000 |

## H NeurIPS Paper Checklist

1. **Claims.** Yes. The abstract and introduction present MapShift as an executable post-intervention protocol and benchmark. The empirical claims cover the 24-motif reference study and held-out ProcTHOR navigation, with the specified observation and planning interfaces.
2. **Limitations.** Yes. We discuss the scope and limitations of the results in Section 7.
3. **Theory, assumptions, and proofs.** N/A. We define an evaluation protocol and the quantities it measures, but don't present theorems that require proof.
4. **Experimental result reproducibility.** Yes. The methodology, artifact, and appendix sections explain the versioned configurations, task generation, baseline training, and grouped bootstrap estimation. For the 3D study, Section 4.5 gives the held-out house split, certification counts, observations, and model seeds.
5. **Open access to data and code.** Yes. The submission provides executable benchmark code, versioned configurations, schemas, and reproduction commands for the reference study. Section 4.5 describes the 3D study and its evaluation requirements.
6. **Experimental setting/details.** Yes. We specify the splits, exploration budget, intervention families and severities, task classes, baseline mechanisms, and model seeds. We also describe certification and score aggregation for the reference and ProcTHOR studies.
7. **Experiment statistical significance.** Yes. We report grouped-bootstrap intervals for the reference study and paired 95% intervals clustered by held-out house for ProcTHOR, keeping learned-system seeds paired within each house.
8. **Experiments compute resources.** Yes. Appendix B describes the CPU smoke test and the full study on a single CUDA GPU, with roughly 30–36 wall-clock hours budgeted on an NVIDIA L4 GPU. This main compute estimate doesn't include preliminary and debug runs.
9. **Code of ethics.** Yes. We develop a controlled evaluation benchmark without using human-subject data, private data, or high-risk deployed systems.
10. **Broader impacts.** Yes. In the appendix, we discuss the risk of treating benchmark performance as evidence of readiness for real-world autonomy.
11. **Safeguards.** N/A. We release neither a high-risk pretrained model nor a dual-use generative model.
12. **Licenses.** Yes. We release the code under the MIT License, while generated benchmark outputs and rendered results are released under CC BY 4.0. Third-party dependencies and tools are listed in `THIRD_PARTY_LICENSES.md`.
13. **Assets.** Yes. We document the executable benchmark, its output schema, limitations, and intended use. The paper introduces no static dataset; generated outputs hosted as datasets should include Croissant/RAI metadata.
14. **Crowdsourcing and human subjects.** N/A. Neither crowdsourcing nor human subjects are used in this study.
15. **IRB approvals.** N/A. The study involves no human-subject research.
16. **Declaration of LLM usage.** N/A. We don't use LLMs in the benchmark methodology, baselines, or experimental results.